

UNIVERSIDAD EAFIT
ESCUELA DE CIENCIAS APLICADAS E INGENIERÍA

Editor de Texto Interactivo CLI en C sobre la Interfaz de Llamadas al Sistema POSIX

Documento de sustentación técnica

Asignatura

Sistemas Operativos (SO2026B)

Evaluación

Evaluación práctica — Primer parcial
Equipo de tres integrantes

Integrantes

- Jhon Jairo Pulgarín Restrepo - Yesid Hurtado Montoya - Andrés Felipe Eusse

Profesor

Edison Valencia

Índice

1. Resumen	2
2. Compilación y uso	2
2.1. Objetivos del Makefile	2
2.2. Las dos formas de invocar el editor	2
2.3. Comandos del editor	3
2.4. Comandos del shell, por categoría	3
3. Análisis del shell de clase	3
3.1. El registro de comandos es una tabla de datos	4
3.2. El contrato del handler no impone duración	4
3.3. Las categorías clasifican por modelo de interacción	4
4. La decisión de integración	4
4.1. Primera pregunta: categoría existente o nueva	4
4.2. Segunda pregunta: en proceso o proceso aislado	5
4.3. Lo que la integración no necesitó tocar	5
4.4. Un defecto del código base que la integración sacó a la luz	5
5. Estructuras de datos seleccionadas	6
5.1. Representación del archivo: sin índice de líneas en memoria	6
5.2. Estado de la sesión	6
5.3. El portapapeles secuencial	7
5.4. Interpretación de y y x	7
6. Implementación: cómo se mueven los bytes	7
6.1. Borrado de una línea (d)	7
6.2. Inserción (i) y pegado (x)	7
6.3. Añadido (a) y la línea incompleta	8
6.4. Búsqueda (s): el costo cuadrático evitado	8
6.5. fstat y no stat	8
6.6. Mezclar printf y write: un riesgo real	8
6.7. Analizador de comandos propio	8
6.8. Un solo editor, dos puntos de entrada	8
7. Manejo de errores	9
8. Validación	9
9. Conclusiones	10
A. Requisitos cubiertos	10
B. Mapa de archivos	11

1. Resumen

Editor de texto interactivo en C, operado por línea de comandos, cuyo acceso a disco se realiza únicamente con llamadas al sistema POSIX: `open`, `read`, `write`, `lseek`, `ftruncate`, `fstat` y `close`. No se usa ninguna función de alto nivel de la biblioteca estándar (`fopen`, `fread`, `fwrite`, `fclose`) sobre el archivo editado.

El editor se integró en el shell de la asignatura creando una **categoría nueva**, edición. Esa decisión es el eje del documento y se justifica en la Sección 4.

Dónde se aborda cada criterio de la rúbrica

Criterio	Peso	Secciones
Implementación técnica y pensamiento crítico	40 %	3, 4, 5 y 6
Complejidad según tamaño del equipo	30 %	2, 8 y Anexo A
Manejo de errores	15 %	7
Entregables y documentación	15 %	2, 8 y Anexo B

2. Compilación y uso

Requiere un entorno Linux con `gcc` y `make`. En Windows, WSL.

```
cd shell
make          # compila los dos binarios: eafitOS (shell) y editor
./eafitOS     # lanza el shell
eafitOS> edit notas.txt
```

2.1. Objetivos del Makefile

Objetivo	Acción
<code>make / all</code>	Compila <code>eafitOS</code> y <code>editor</code> . No ejecuta nada
<code>make run</code>	Compila y lanza el shell interactivo
<code>make run-editor</code>	Compila y lanza el editor independiente
<code>make test</code>	Ejecuta la batería de 46 pruebas
<code>make asan</code>	Las mismas pruebas con AddressSanitizer, UBSan y LeakSanitizer
<code>make valgrind</code>	Las mismas pruebas bajo valgrind, si está instalado
<code>make clean</code>	Borra binarios, objetos y residuos de pruebas

El objetivo `all` no ejecuta ningún binario de forma deliberada. El Makefile original lanzaba el shell al terminar de enlazar, lo que impedía usar `make` dentro de un guion de pruebas; ejecutar es responsabilidad de `run`.

2.2. Las dos formas de invocar el editor

Hay **un solo editor**. El archivo `editor.c` se compila tanto dentro del shell como dentro del binario independiente, de modo que ambos ejecutan exactamente el mismo código (véase la Sección 6.8).

Forma	Desde dónde	Para qué
<code>edit <archivo></code>	Dentro de <code>./eafitOS</code>	La integración que se evalúa
<code>./editor <archivo></code>	Desde la terminal	Probar el editor solo; lo usa <code>pruebas.sh</code>

Al entrar, el indicador cambia de `eafitOS>` a `editor[archivo]>`; al salir con `q` se regresa al shell, que sigue vivo.

2.3. Comandos del editor

Comando	Descripción	Llamadas al sistema
<code>o <archivo></code>	Abre el archivo; lo crea con modo 0644 si no existe	<code>open</code> , <code>close</code>
<code>p</code>	Imprime todo el archivo recorriendo bytes	<code>lseek</code> , <code>read</code> , <code>write</code>
<code>p <n></code>	Imprime únicamente la línea <i>n</i>	<code>lseek</code> , <code>read</code> , <code>write</code>
<code>a <texto></code>	Añade el texto como nueva última línea	<code>lseek</code> , <code>write</code>
<code>d <n></code>	Borra la línea <i>n</i> , desplaza la cola y trunca	<code>read</code> , <code>write</code> , <code>lseek</code> , <code>ftruncate</code>
<code>i <n> <texto></code>	Inserta en la línea <i>n</i> desplazando el resto	<code>lseek</code> , <code>read</code> , <code>write</code>
<code>s <palabra></code>	Lista las líneas que contienen la palabra	<code>read</code>
<code>m</code>	Metadatos del inodo	<code>fstat</code>
<code>y <n></code>	Copia la línea <i>n</i> al portapapeles	<code>lseek</code> , <code>read</code>
<code>x <n></code>	Pega el portapapeles en la línea <i>n</i>	<code>lseek</code> , <code>read</code> , <code>write</code>
<code>q</code>	Cierra el descriptor y sale sin fugas	<code>close</code>

Las líneas se numeran desde 1. El texto de `a` e `i` se toma literal hasta el fin de la línea, con o sin comillas. Como extensiones documentadas, fuera del requisito mínimo: `y` sin argumentos muestra el portapapeles, `y -` lo vacía, `t` activa o desactiva la traza de llamadas al sistema y `h` muestra la ayuda.

2.4. Comandos del shell, por categoría

El shell conserva sus cuatro categorías originales y suma la quinta:

Categoría	Comandos	Syscalls que demuestra
datos	<code>d_create</code> , <code>d_read</code> , <code>d_info</code> , <code>d_copy</code>	<code>open</code> , <code>read</code> , <code>write</code> , <code>stat</code>
memoria	<code>m_sbrk</code> , <code>m_mmap</code> , <code>m_info</code>	<code>sbrk</code> , <code>mmap</code> , <code>munmap</code>
monitoreo	<code>p_fork</code> , <code>p_exec</code> , <code>p_kill</code> , <code>p_monitor</code>	<code>fork</code> , <code>execvp</code> , <code>waitpid</code> , <code>kill</code>
edicion	<code>edit</code>	<code>open</code> , <code>lseek</code> , <code>ftruncate</code> , <code>fstat</code>
utilidades	<code>saludar</code> , <code>despedir</code> , <code>hora</code> , <code>fecha</code>	<code>getuid</code> , <code>time</code>

La ayuda interactiva responde a `help`, `help <categoria>` y `help <comando>`. La lista de categorías se deriva automáticamente de la tabla de comandos (Sección 4.4).

3. Análisis del shell de clase

Decidir cómo integrar el editor exigía primero entender qué clase de programa es el shell. El análisis arrojó tres hallazgos que condicionaron todo lo demás.

3.1. El registro de comandos es una tabla de datos

El shell no enruta con una cadena de condicionales, sino con una tabla de estructuras declarada en `shell.h`. Añadir un comando significa, por diseño, añadir una fila.

```
typedef struct {
    const char *name;           /* nombre que escribe el usuario */
    const char *category;       /* datos, memoria, monitoreo, ... */
    const char *usage;          /* sintaxis para la ayuda */
    const char *description;    /* que hace el comando */
    const char *syscalls;       /* syscalls que demuestra */
    int (*handler)(int argc, char **argv);
} Command;
```

3.2. El contrato del handler no impone duración

Este es el hallazgo decisivo. La firma `int (*handler)(int, char**)` no restringe cuánto puede tardar un handler ni qué puede hacer mientras tanto. Los catorce comandos originales retornan en microsegundos, lo que sugiere que un handler *debe* ser instantáneo; pero nada en el contrato lo obliga: puede leer de STDIN y ciclar indefinidamente.

Esa observación es la que permite alojar un editor completo dentro del shell sin modificar su núcleo.

3.3. Las categorías clasifican por modelo de interacción

Las cuatro categorías originales parecen agrupar por familia de llamadas al sistema. Al leerlas en conjunto, sin embargo, aparece una invariante más fuerte y no declarada:

Todos los comandos del shell son de un solo disparo y sin estado: ejecutan, imprimen el resultado y retornan. Ninguno conserva memoria entre invocaciones, ninguno mantiene un descriptor abierto y ninguno lee de STDIN.

Ese, y no la familia de syscalls, es el criterio real de clasificación. Reconocerlo permitió decidir la integración de forma argumentada en lugar de por analogía superficial.

4. La decisión de integración

El enunciado pide determinar si el editor se clasifica en una categoría existente o exige una nueva, y justificarlo. El problema se separó en dos preguntas independientes: dónde se clasifica y cómo se ejecuta.

4.1. Primera pregunta: categoría existente o nueva

La opción inmediata es **datos**: el editor usa `open`, `read` y `write`, igual que `d_create` o `d_copy`. Ese razonamiento clasifica por **familia de syscalls** y se descartó, porque no es el criterio que el shell aplica en realidad.

El editor rompe la invariante de las cuatro categorías en tres puntos simultáneos:

1. Mantiene un descriptor abierto a lo largo de decenas de comandos sucesivos.
2. Conserva estado en memoria dinámica durante toda la sesión: el portapapeles.
3. Se apropia de STDIN con su propio sub-REPL anidado en el del shell.

La diferencia no es de familia de syscalls sino de **modelo de interacción**. Clasificarlo en **datos** obligaría a que esa categoría significara dos cosas a la vez, degradando el valor pedagógico de toda la clasificación.

Decisión: se creó la categoría **edicion**, con el comando **edit**. Cada categoría conserva un significado único, y queda abierta la puerta a futuras aplicaciones interactivas del mismo tipo sin reabrir la discusión.

4.2. Segunda pregunta: en proceso o proceso aislado

Quedaba decidir si el editor debía correr dentro del proceso del shell o como binario externo lanzado con **fork** y **execvp**.

	En proceso (elegido)	Aislado (descartado)
Mecanismo	Llamada a función	fork + execvp + waitpid
Costo de arranque	Nulo	Un fork y un exec por invocación
Tabla de descriptors	La del propio shell	Propia; el shell no ve el archivo
Fallo grave del editor	Se lleva por delante al shell	Sólo muere el hijo
Decisión que exige	Encajar un sub-REPL en el contra- to del handler	Ninguna

El argumento decisivo es que el shell *ya ofrece* **p_exec**, que hace exactamente **fork** más **execvp** sobre cualquier binario. Integrar el editor por esa vía no habría requerido ninguna decisión de diseño: habría sido escribir **p_exec ./editor** con otro nombre. Además habría desperdiciado la propiedad más interesante desde el punto de vista pedagógico: al correr en proceso, el editor abre el archivo en la **misma tabla de descriptors del shell**, comprobable en vivo con **m_info** antes y después.

4.3. Lo que la integración no necesitó tocar

La mejor evidencia de que la decisión respeta la arquitectura existente es el tamaño del cambio: registrar el editor requirió **ocho líneas**, un prototipo en **shell.h** y una entrada en la tabla **commands[]** de **main.c**, con el mismo formato que las catorce existentes.

```
{
    "edit", "edicion",
    "edit [archivo]",
    "Abre el editor de texto CLI dentro de este mismo proceso.",
    "open(2), read(2), write(2), lseek(2), ftruncate(2), fstat(2), close(2)",
    cmd_edit
},
```

Quedaron **intactos** el bucle REPL principal, el tokenizador **parse_line()**, la estructura **Command**, la firma del handler y los catorce comandos originales. Que una aplicación interactiva con estado propio quepa sin modificar el núcleo del shell demuestra que la extensión respeta su arquitectura en lugar de forzarla.

4.4. Un defecto del código base que la integración sacó a la luz

Al añadir la categoría apareció un problema real en el shell original: **print_help()** traía las cuatro categorías escritas a mano en una cadena de **strcmp**, y era el único punto del shell que **no** se alimentaba de la tabla de comandos.

```

/* Antes: unico punto no gobernado por la tabla */
if (strcmp(arg, "datos") == 0 || strcmp(arg, "memoria") == 0 ||
    strcmp(arg, "monitoreo") == 0 || strcmp(arg, "utilidades") == 0) { ... }

/* Ahora: la existencia de una categoria se deduce recorriendo commands[] */
if (es_categoria(arg)) { ... }

```

Bastaba añadir un quinto `strcmp` y seguir, pero eso dejaba la trampa puesta para quien añadiera la siguiente categoría. Con `es_categoria()` y `listar_categorias()`, registrar un comando con una categoría nueva basta para que la ayuda funcione sola.

5. Estructuras de datos seleccionadas

5.1. Representación del archivo: sin índice de líneas en memoria

Decisión: el archivo en disco es la única fuente de verdad. No se mantiene ningún arreglo de desplazamientos de línea en memoria.

La alternativa habitual es cachear el desplazamiento donde comienza cada línea, lo que da acceso $O(1)$ a la línea n . Se descartó porque cada mutación (`a`, `i`, `d`, `x`) invalida todos los desplazamientos posteriores, y mantener ese índice sincronizado con el disco es la principal fuente de corrupción silenciosa. El fallo típico no es una violación de segmento, que se detecta de inmediato, sino un archivo mal escrito que sólo se descubre mucho después.

	Índice en memoria	Recorrido byte a byte (elegido)
Localizar la línea n	$O(1)$	$O(n)$
Riesgo de desincronización	Alto, en cada mutación	Nulo por construcción
Memoria adicional	Un <code>off_t</code> por línea	Un búfer fijo de 4096 B
Complejidad del código	Alta (invalidación)	Baja

A esta escala el costo $O(n)$ es irrelevante y a cambio se elimina una clase entera de errores. La decisión prioriza **corrección demostrable sobre rendimiento no necesario**.

Matiz sobre “byte a byte”: la expresión describe la lógica, no la granularidad de las syscalls. El editor lee en bloques de 4096 bytes con `read` y escanea el bloque ya en memoria buscando el fin de línea. Emitir una syscall por byte sería correcto pero absurdo: cada byte costaría una transición al modo kernel. Es la razón por la que existe el almacenamiento intermedio de `stdio`.

5.2. Estado de la sesión

Todo el estado vive en una estructura que se pasa por referencia a cada comando. No hay variables globales, lo que permite que el editor funcione igual dentro del shell que como binario suelto.

```

typedef struct {
    int      fd;                /* descriptor abierto, -1 si no hay */
    char     ruta[ED_RUTA_MAX]; /* ruta del archivo actual          */
    int      abierto;           /* 1 si hay archivo abierto         */
    int      trazar;            /* traza de syscalls activada       */
    Portapapeles pp;           /* portapapeles secuencial          */
} Editor;

```

5.3. El portapapeles secuencial

Estructura elegida: arreglo dinámico de cadenas (`char **`) con crecimiento por duplicación de capacidad mediante `realloc`.

Se descartó la lista enlazada: el portapapeles se recorre siempre completo y en orden al pegar, y nunca se inserta ni elimina por el medio, que es justamente el caso en que la lista gana. El arreglo ofrece localidad de memoria y liberación trivial; la lista pagaría un puntero por línea sin aportar nada.

El calificativo *secuencial* se interpretó así: **y acumula** en lugar de reemplazar, lo que permite recoger varias líneas sueltas y pegarlas juntas como un bloque con una sola **x**. El portapapeles no se consume al pegar y se vacía con **y -**.

5.4. Interpretación de **y** y **x**

El enunciado enumera “Copiar y Pegar: **y** [**n**] y **x** [**n**]” sin asignar cada verbo a cada letra. En **vi** la letra **p** corresponde a *paste*, pero aquí **p** ya está tomada por *print*, que es un comando base obligatorio. Se adoptó la convención coherente con un portapapeles secuencial: **y** **<n>** **copia** la línea **n** y la añade al final del portapapeles; **x** **<n>** **pega** todo el portapapeles a partir de la línea **n**.

6. Implementación: cómo se mueven los bytes

Un archivo no ofrece una operación “borrar en el medio” ni “insertar en el medio”. El sistema de archivos sólo permite leer, escribir en una posición y recortar el tamaño total. Compactar o abrir hueco es responsabilidad del programa.

6.1. Borrado de una línea (d)

```

      inicio      cola_inicio
[ cabeza ][ línea n a borrar ][ ..... cola ..... ]
      |<-- ocupa -->|

1. localizar el inicio de la línea n y cuantos bytes ocupa (texto + \n)
2. reservar con malloc un bufer del tamaño de la cola
3. leer la cola completa a memoria
4. lseek(inicio) + write(colas) -> la cola se desplaza hacia atrás
5. ftruncate(inicio + cola_len) -> se recorta el residuo del final

```

Dos detalles son errores clásicos si se omiten, y ambos tienen prueba dedicada:

- **La cola debe leerse completa a memoria antes de escribir nada.** Las regiones de origen y destino se solapan; escribir mientras se lee destruiría bytes aún no leídos.
- **ftruncate es obligatorio.** Sin él quedaría al final un residuo duplicado de la cola, porque `write` sobrescribe pero nunca acorta. La prueba “borrar hasta vaciar” exige cero bytes exactos, condición que sólo se cumple si el truncado se aplica.

6.2. Inserción (i) y pegado (x)

La inserción es la operación simétrica: leer la cola, escribir el texto nuevo y reescribir la cola detrás. **No** necesita `ftruncate` porque el archivo sólo crece.

El pegado usa el mismo mecanismo con una diferencia deliberada: arma primero en memoria **un solo búfer** con todas las líneas del portapapeles y hace **una sola** escritura. Pegar línea por línea obligaría a releer y desplazar la cola una vez por línea, con costo cuadrático.

Se admite insertar en la posición $total + 1$, con lo que `i` degenera en `a` sin necesitar un caso especial.

6.3. Añadido (a) y la línea incompleta

Añadir al final es la mutación más barata: `lseek(SEEK_END)` y escribir, sin memoria dinámica. Hay un caso borde que rompe la semántica si se ignora: un archivo cuya última línea **no** termina en salto de línea. Añadir directamente pegaría el texto nuevo al final de la línea existente. El editor lee el último byte y escribe el salto que falta antes de continuar.

6.4. Búsqueda (s): el costo cuadrático evitado

La implementación ingenua consultaría la línea k para $k = 1 \dots N$ con la rutina de localización; como cada consulta recorre el archivo desde el principio, el costo sería $O(N^2)$.

La implementación adoptada recorre el archivo **una sola vez** en bloques, acumulando la línea en curso en un búfer dinámico que crece por duplicación con `realloc`. El costo es $O(N)$, y es el punto donde el proyecto demuestra el manejo de búferes dinámicos que exige el enunciado para equipos de dos o más integrantes.

6.5. `fstat` y `no stat`

El comando de metadatos usa `fstat` sobre el descriptor abierto, no `stat` sobre la ruta. Si alguien renombra o elimina el archivo mientras el editor lo tiene abierto, `stat(ruta)` fallaría o describiría un archivo distinto, mientras que `fstat(fd)` sigue describiendo el inodo que el editor está editando. **Un descriptor es una referencia al inodo, no al nombre.** El shell base usa `stat` en `d.info`, y allí es la elección correcta porque no hay ningún archivo abierto.

6.6. Mezclar `printf` y `write`: un riesgo real

El enunciado exige emitir el contenido con `write` sobre el descriptor 1, mientras que los números de línea y los mensajes se imprimen con `printf`. No comparten camino: `printf` escribe en un búfer de espacio de usuario que `stdio` vacía cuando le conviene, y `write` va directo al kernel. Mezclarlos sin cuidado hace que el contenido aparezca **antes** que los mensajes, aunque el código diga lo contrario. Por eso toda la salida pasa por una rutina que fuerza `fflush(stdout)` antes de cada `write`.

6.7. Analizador de comandos propio

El editor **no** reutiliza `parse_line()` del shell. Ese analizador parte por espacios y exige comillas para agrupar: correcto para un shell, inadecuado para un editor, donde obligaría a escribir a `"hola mundo"` y volvería imposible añadir una línea con comillas dobles. El analizador del editor separa sólo el comando (y el número, cuando lo lleva) y toma el resto de la línea literalmente, que es el criterio de `ed`.

6.8. Un solo editor, dos puntos de entrada

`editor.c` se compila tanto dentro del shell como dentro del binario independiente; `main_editor.c` contiene únicamente la función `main` y el manejo de argumentos. No son dos programas parecidos: son el mismo programa. Corregir un defecto en `editor.c` lo corrige en ambos, y es lo que permite que la batería de pruebas ejercite el binario suelto con la garantía de validar exactamente el código que corre dentro del shell.

7. Manejo de errores

Toda llamada al sistema verifica su valor de retorno sin excepción. Los fallos se reportan por la salida de error estándar con **perror**, que añade la descripción textual de **errno**:

```
editor[notas.txt]> o /raiz/prohibido.txt
open: Permission denied
```

Más allá de la verificación básica, el editor atiende cuatro situaciones que suelen pasarse por alto:

Situación	Tratamiento
Transferencias parciales	write y read pueden mover <i>menos</i> bytes de los pedidos sin que sea un error; ignorarlo corrompe archivos en silencio. Todas las transferencias pasan por rutinas que reintentan hasta completar
Interrupción por señal	Esas mismas rutinas reintentan ante EINTR en lugar de darlo por fallo
Validación previa	Los índices fuera de rango se rechazan <i>antes</i> de tocar el disco, de modo que un comando inválido nunca deja el archivo a medio escribir
Liberación única	q y la salida por Ctrl+D recorren el mismo camino: close del descriptor y free del portapapeles. La rutina es idempotente

Los comandos que exigen un archivo abierto avisan al usuario en lugar de fallar, y ningún camino de error deja un descriptor sin cerrar.

8. Validación

El guion **pruebas.sh** alimenta el binario del editor por STDIN con secuencias de comandos y compara **byte a byte** el archivo resultante contra el contenido esperado. La verificación se hace sobre el archivo en disco, no sobre la salida por pantalla: lo que interesa demostrar es que las llamadas al sistema dejaron el archivo como debía quedar.

Bloque	Qué valida
1. Comandos base	o crea, a añade, p imprime todo y por línea, q cierra
2. Borrado	d en el medio, en la última línea y hasta vaciar el archivo
3. Inserción	i en el medio, al principio y en <i>total + 1</i>
4. Búsqueda	Coincidencias múltiples, sin falsos positivos, sin coincidencias
5. Metadatos	m reporta inodo, tamaño, permisos y fecha de modificación
6. Portapapeles	y y x , acumulación, pegado al final, vaciado
7. Casos borde	Ver lista siguiente
8. Permisos	El archivo se crea con modo 0644 ; cierre ordenado
9. Integración	help lista edicion ; edit escribe y devuelve el control

Casos borde cubiertos: archivo vacío; índice cero, negativo o mayor que el total en todos los comandos que reciben número; **archivo cuya última línea no termina en salto de línea**, al añadir y al borrar esa línea; **archivo mayor que el bloque de lectura**, con 400 líneas, que fuerza varias iteraciones de **read** y comprueba que no se pierden líneas partidas entre bloques; **borrado hasta vaciar**, que exige cero bytes exactos; comandos que exigen archivo abierto sin haberlo abierto; comando inexistente; y texto con espacios con y sin comillas.

Verificación	Herramienta	Resultado
Compilación	<code>gcc -Wall -Wextra -std=gnu99</code>	0 errores, 0 advertencias
Funcionalidad	<code>make test</code>	46 de 46 casos
Memoria y comportamiento	<code>make asan</code>	46 de 46, sin hallazgos

El objetivo `asan` recompila el editor con AddressSanitizer, UndefinedBehaviorSanitizer y LeakSanitizer y ejecuta la batería completa: ninguna fuga de memoria, ningún acceso inválido, ningún comportamiento indefinido. Esto es lo que acredita el requisito del enunciado de que `q` salga “sin dejar fugas de memoria”.

Nota sobre el sistema de archivos. La prueba de permisos verifica el modo `0644`. Si el proyecto reside en una unidad de Windows montada en WSL, ese montaje no conserva permisos POSIX y todo aparece como `0777`, lo que haría fallar la prueba sin que el editor tenga defecto alguno. El guion lo detecta y traslada las pruebas a `/tmp`. Se documenta porque es un ejemplo de validación que debe distinguir un fallo del programa de una limitación del entorno.

9. Conclusiones

1. **La clasificación de un comando debe seguir el criterio real de la arquitectura, no la analogía superficial.** El editor comparte llamadas al sistema con `datos`, pero no comparte su modelo de interacción. Reconocer que las categorías del shell agrupan por modelo y no por familia de syscalls es lo que justifica la creación de `edicion`.
2. **Una buena arquitectura se reconoce por lo que no hay que modificar.** El contrato del handler admitió un sub-REPL completo sin cambiar una línea del núcleo del shell; las ocho líneas de integración son la medida objetiva de esa compatibilidad.
3. **Priorizar la corrección demostrable sobre el rendimiento innecesario fue acertado.** Renunciar al índice de líneas costó un factor $O(n)$ irrelevante a esta escala y eliminó la clase de errores más difícil de diagnosticar en un editor.
4. **Las abstracciones del espacio de usuario tienen costos observables.** El almacenamiento intermedio de `stdio`, invisible al trabajar de forma interactiva, dejó inservible a `p_exec` al combinarse con `fork` y supuso un riesgo de orden de salida al mezclarse con `write`.
5. **Extender un sistema obliga a entenderlo, y entenderlo revela sus defectos.** La integración sacó a la luz dos fallos del shell base: una ayuda que no se alimentaba de su propia tabla de comandos y una entrada almacenada que dejaba inservible a `p_exec` al guionarlo. Ambos quedaron corregidos.

A. Requisitos cubiertos

Los requisitos del enunciado son acumulativos: un equipo de tres integrantes implementa los comandos base, los de equipos de dos y los propios de su tamaño.

Nivel	Requisito	Estado
Base	<code>o</code> , <code>p</code> , <code>a</code> , <code>d</code> , <code>q</code>	Cumplido
Base	Integración en el shell de clase	Cumplido
Equipos de 2	<code>i <n> <texto></code> : inserción arbitraria	Cumplido
Equipos de 2	<code>s <palabra></code> : búsqueda simple	Cumplido
Equipos de 2	Búferes dinámicos con <code>malloc</code> , <code>realloc</code> y <code>free</code>	Cumplido
Equipos de 3	<code>m</code> : tamaño, permisos, inodo y modificación vía <code>fstat</code>	Cumplido
Equipos de 3	<code>y <n></code> copiar y <code>x <n></code> pegar, portapapeles secuencial	Cumplido

No corresponden a este equipo los requisitos de cuatro integrantes (deshacer y rehacer con archivos temporales en `/tmp` y `unlink`) ni los de cinco (múltiples búferes y bloqueo de archivos con `fcntl`).

B. Mapa de archivos

Archivo	Contenido
<code>editor.h</code>	Interfaz del módulo: decisiones de diseño y <code>editor_main()</code>
<code>editor.c</code>	Todo el editor. El resto de funciones son <code>static</code>
<code>main_editor.c</code>	Sólo la función <code>main</code> del binario independiente
<code>shell.h</code>	Cabecera del shell y prototipo de la categoría <code>edicion</code>
<code>main.c</code>	Tabla de comandos, ayuda derivada de la tabla y REPL del shell
<code>cat_*.c</code>	Comandos de las cuatro categorías originales, sin modificar
<code>pruebas.sh</code>	Batería de 46 pruebas automatizadas
<code>Makefile</code>	Objetivos de compilación, ejecución y verificación